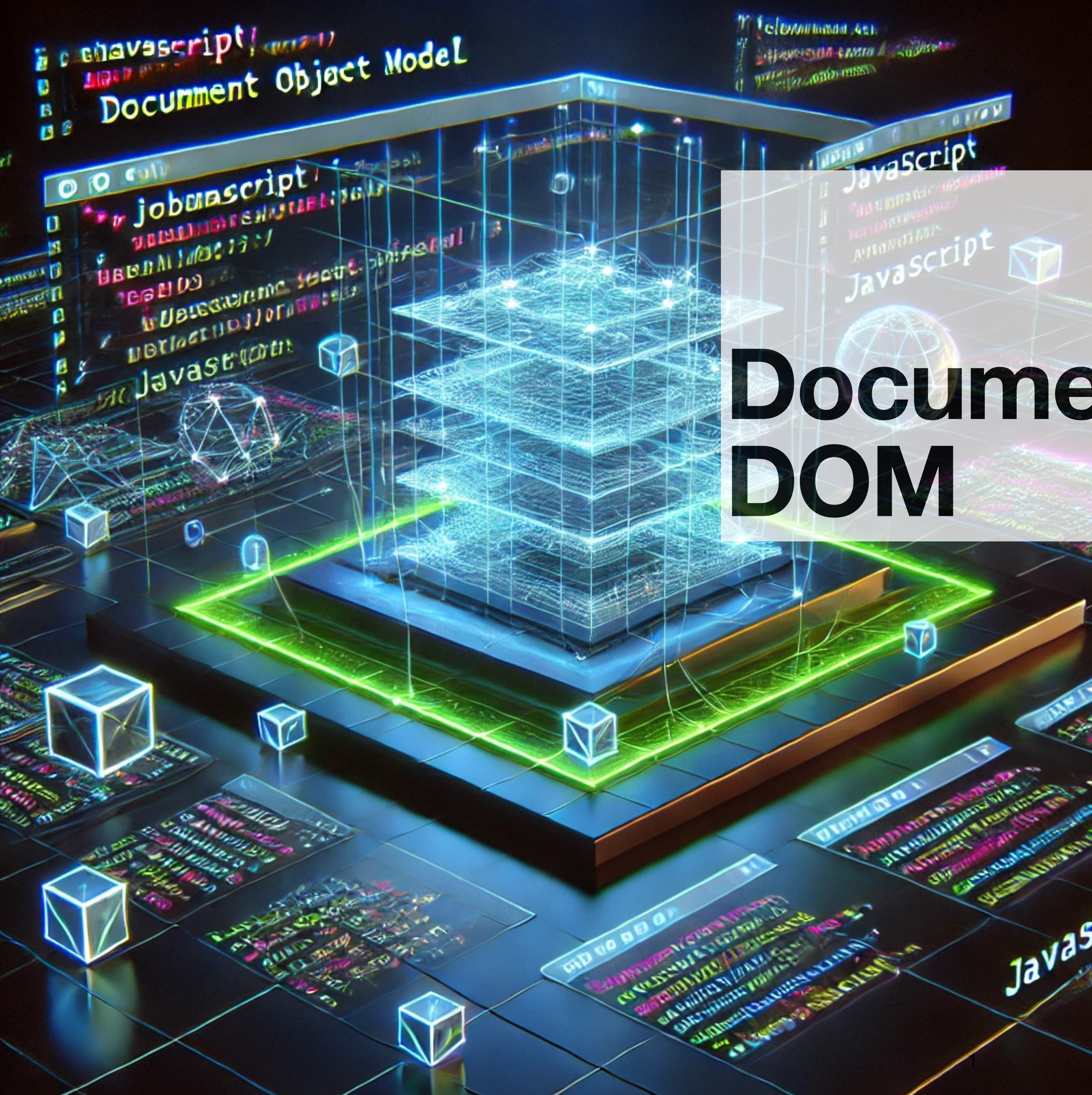




Document Object Model DOM

Referência:

JavaScript - Guia do Programador - MAUJOR

<http://www.w3c.br/cursos/html5>

<http://www.devmedia.com.br>

DOM (Document Object Model)

- É uma representação em forma de árvore do documento HTML.
- Cada elemento HTML se torna um nó (node) da árvore, que pode ser manipulado dinamicamente com JavaScript.

Por que estudar DOM?

- Permite atualizar o conteúdo de uma página sem recarregá-la.
- Facilita interatividade (cliques, digitação, efeitos visuais).
- Base para bibliotecas e frameworks como React, Vue e Angular.

O que é o DOM?

A ponte entre HTML e JavaScript

Definição

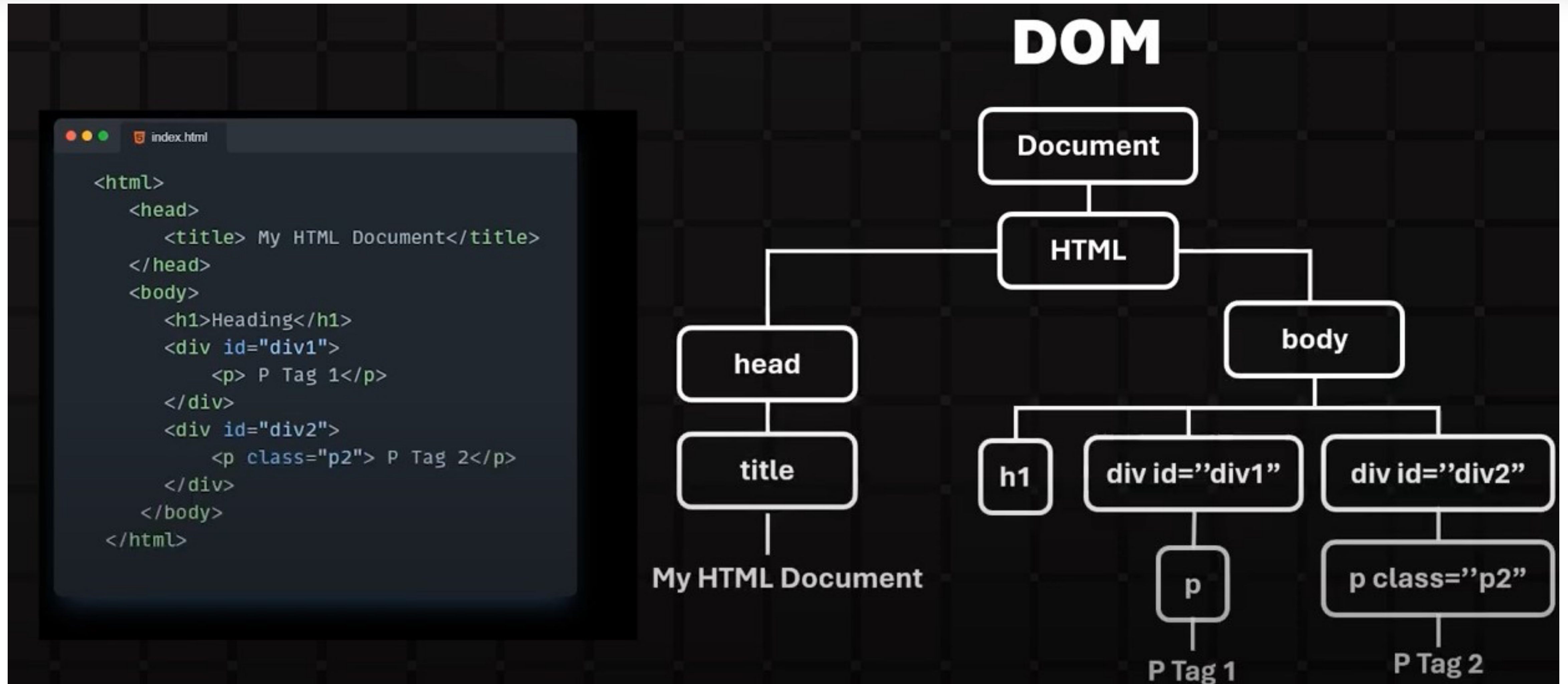
DOM (Document Object Model) é uma API padronizada pelo W3C que representa o documento HTML como uma **árvore de objetos** na memória do navegador.

Cada tag HTML vira um **nó (node)** que pode ser lido, alterado, criado ou removido dinamicamente via JavaScript — sem recarregar a página.

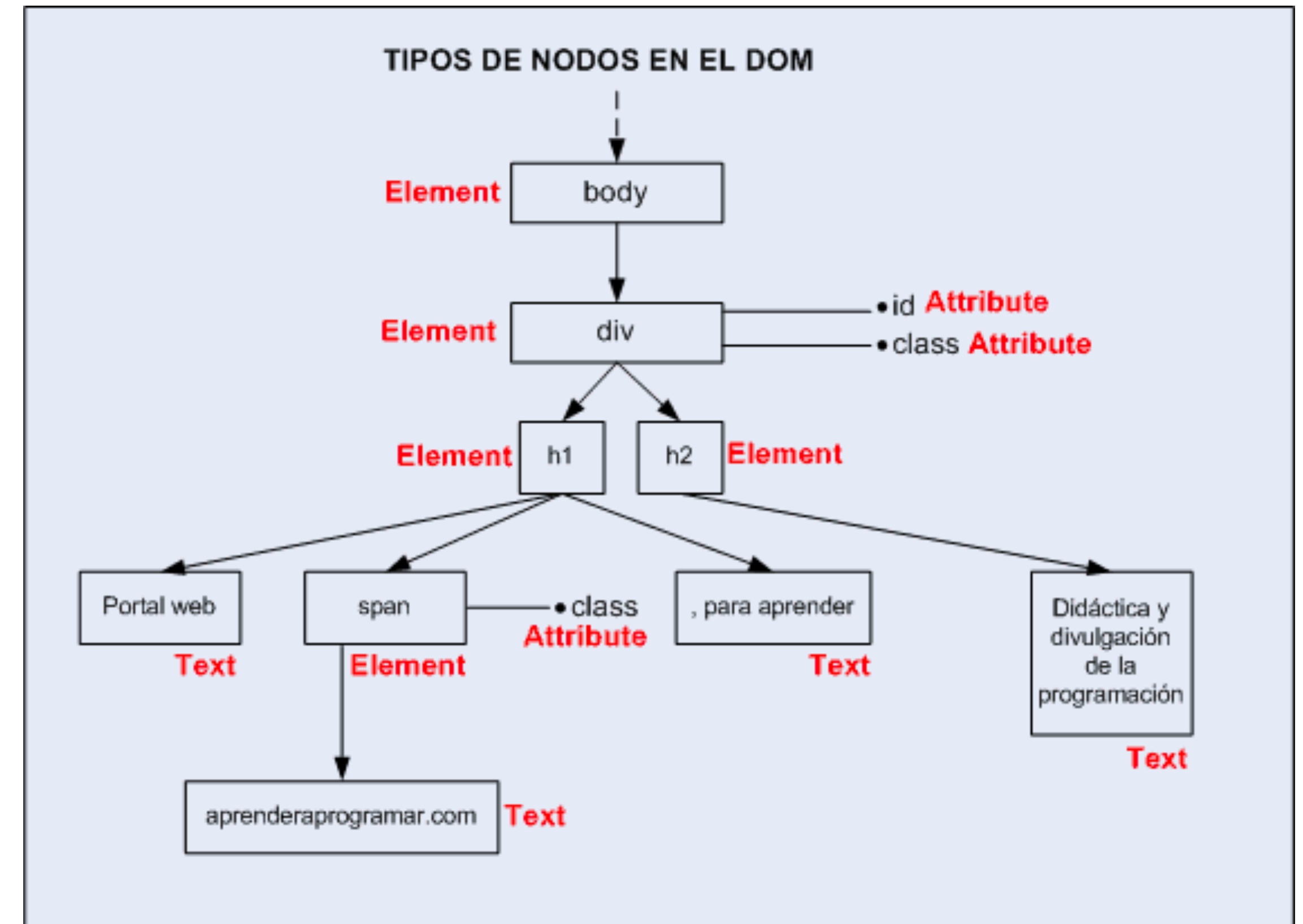
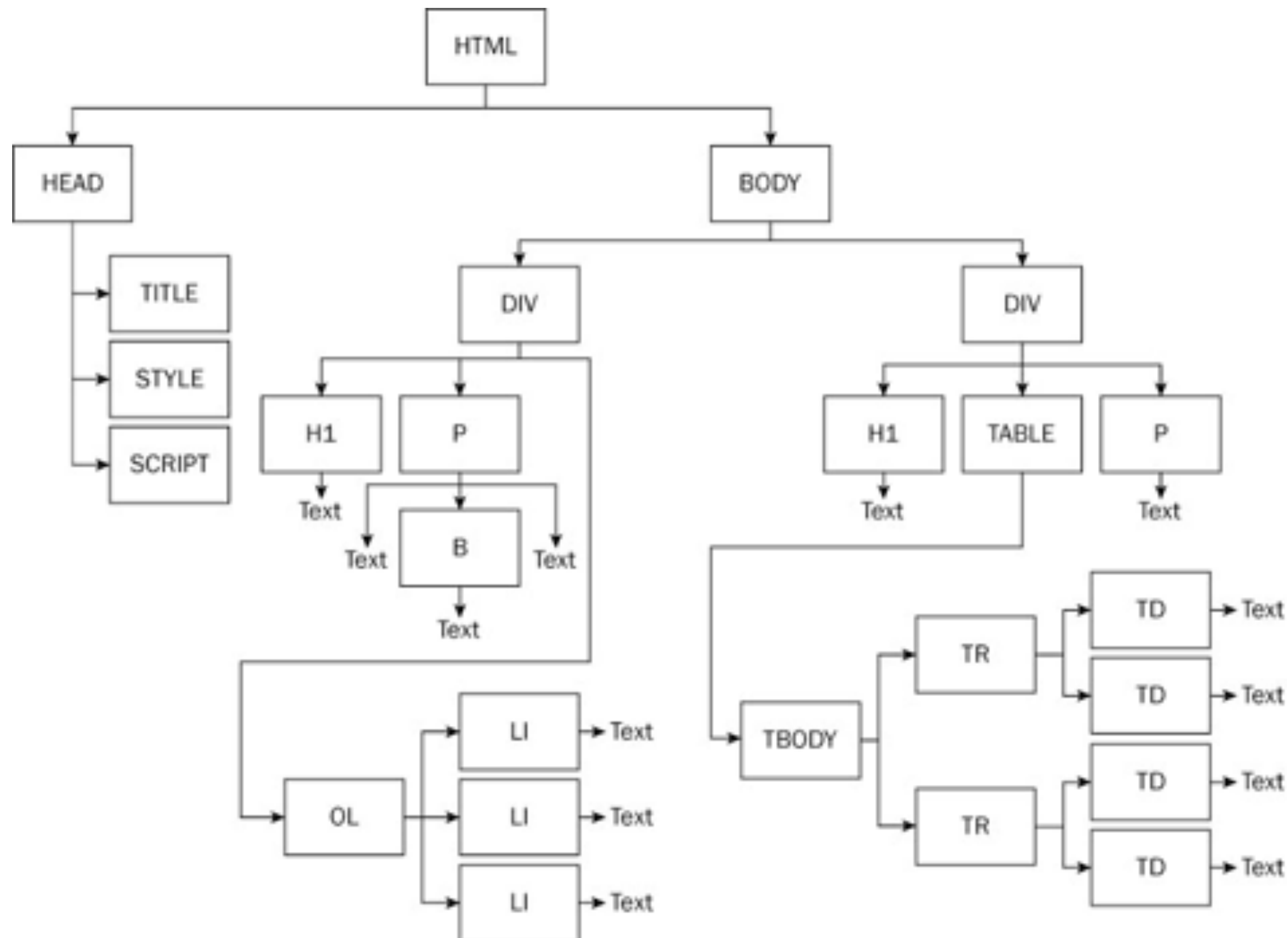
Por que estudar DOM?

- Páginas interativas sem recarregar
- Base de React, Vue e Angular
- Validação de formulários em tempo real
- Animações e efeitos visuais
- SPAs (Single Page Applications)
- Extensões de navegador

Associando...



Árvore DOM



Live DOM Viewer

Live DOM Viewer

Markup to test ([permalink](#), [save](#), [upload](#), [download](#), [hide](#)):

```
<head></head>
<body>
  <h1>Um elemento H1</h1>
  <div>Uma DIV</div>
  <p>Um elemento p com um elemento <span style="color: brown;">span</span>dentro.</p>
</body>
</html>
```

DOM view ([hide](#), [refresh](#)):

```
DOCTYPE: html
HTML
  HEAD
  #text:
  BODY
    #text:
    H1
      #text: Um elemento H1
    #text:
    DIV
      #text: Uma DIV
    #text:
    P
      #text: Um elemento p com um elemento
      SPAN style="color: brown;"
        #text: span
      #text: dentro.
    #text:
```

Rendered view: ([hide](#)):

Um elemento H1

Uma DIV

Um elemento p com um elemento spandentro.

Exibir um menu : ([show](#), [refresh](#)):

Teste com esse código:

```
<ul>
  <li>Item 1
  <li>Item 2
</ul>
<p>Um parágrafo <div>com um div dentro</div>
  e mais texto.</p>
```

<https://software.hixie.ch/utilities/js/live-dom-viewer/>

Principais Objetos do DOM

- **Document:** É o objeto raiz do DOM, que representa o próprio documento HTML. Por meio dele, podemos acessar e manipular todos os elementos e nós do documento, como as tags HTML, atributos, textos e até scripts incorporados.
- **Node (nó):** Representa a base de qualquer objeto no DOM, servindo como uma nomenclatura genérica. Cada elemento, atributo ou texto em um documento HTML é um tipo de nó. Existem diferentes tipos de nós, como elemento, atributo, texto e comentário.
- **Element (nó de elemento):** Este tipo de nó representa as tags do HTML, como `<div>`, `<p>`, `<a>`, etc. Os elementos podem ter "nós-filhos", que são os textos e atributos contidos dentro deles. Esse tipo de objeto é o que possibilita a estrutura hierárquica do DOM, permitindo a criação de árvores de elementos aninhados.

Principais Objetos do DOM

- **Attribute (nó de atributo):** Um nó do tipo atributo representa um atributo específico de um elemento HTML, como **id**, **class**, ou **href**. Apesar de serem parte do elemento, no DOM eles podem ser acessados e manipulados individualmente.
- **Text (nó de texto):** O nó de texto representa o **conteúdo textual** que está **dentro das tags HTML**. Ele é o texto propriamente dito, localizado entre as tags e formando o conteúdo exibido na página.

Acesse o DOM depois de carregar toda a página

```
window.onload = function() { console.log("Está carregado!") }
```

```
<body onload="window.alert('Welcome to my home page!');"></body>
```

```
document.onload(funcaoPaginaCarregada())  
function funcaoPaginaCarregada() {  
    // o que fazer após a página ser carregada  
}
```

```
document.addEventListener("DOMContentLoaded", function(event)  
{  
    console.log("Somente serei executado quando o navegador  
terminar  
de analisar todo o DOM");  
});
```

Acessando Elementos no DOM

Métodos de seleção:

```
document.getElementById("id")  
document.getElementsByClassName("classe")  
document.getElementsByTagName("tag")  
document.querySelector("seletor")  
document.querySelectorAll("seletor")
```

```
<p id="intro" class="destaque">Texto introdutório</p>  
<script>  
  const el1 = document.getElementById("intro"); // acesso direto por ID  
  const el2 = document.querySelector(".destaque"); // acesso por seletor CSS  
</script>
```

Dica: querySelector permite usar seletores CSS como #id, .class, tag[atributo], div > p ...

Comparativo entre métodos de seleção DOM

Método	Retorna	Tipo	Observação
getElementById	1 elemento	Elemento	Mais rápido para ID único
getElementsByClassName	HTMLCollection	Live	Lista dinâmica de classes
getElementsByTagName	HTMLCollection	Live	Lista dinâmica por tag
getElementsByName	NodeList	Estático	Usado com forms
querySelector	1 elemento	Elemento	Aceita seletores CSS
querySelectorAll	NodeList	Estático	Lista baseada em CSS, não atualiza

Quando usar cada método?

getElementById: Use quando o elemento tem ID único

getElementsByClassName: Use para obter vários elementos com a mesma classe

querySelector: Use para pegar o primeiro que bate com seletor CSS

querySelectorAll: Use para pegar todos que batem com o seletor CSS

Manipulando Conteúdo e Atributos

Propriedades comuns:

```
el.textContent // texto sem HTML
el.innerHTML // texto com HTML
el.value // valor de campos de formulário
el.setAttribute("atributo", "valor")
el.removeAttribute("atributo")
```

Exemplo:

```
<p id="texto">Original</p>
<button onclick="alterar()">Mudar</button>
<script>
  function alterar() {
    const p = document.getElementById("texto");
    p.textContent = "Texto alterado dinamicamente!";
  }
</script>
```

Estilizando Elementos

Uso do style

```
el.style.color = "red";  
el.style.fontWeight = "bold";
```

Exemplo completo:

```
<p id="info">Texto</p>  
<button onclick="destacar()">Destacar</button>  
<style>  
  .destaque {  
    background-color: yellow;  
    padding: 10px;  
  }  
</style>  
<script>  
  function destacar() {  
    document.getElementById("info").classList.toggle("destaque");  
  }  
</script>
```

Uso de classList:

```
el.classList.add("ativo");  
el.classList.remove("oculto");  
el.classList.toggle("destaque");
```

Curiosidade... benchmarks de desempenho

- className

```
let element = document.getElementById("foo");
let i = 1000;
while (i--) {
    element.className = "bar";
}
```

- classList

```
let element = document.getElementById("foo");
let i = 1000;
while (i--) {
    element.classList.add("bar");
}
```

- setAttribute

```
let element = document.getElementById("foo");
let i = 1000;
while (i--) {
    element.setAttribute("class", "bar");
}
```

Run the benchmark: className vs. setAttribute vs. classList

Eventos no DOM

Sintaxe comum:

```
el.onclick = funcao;  
el.addEventListener("evento", funcao);
```

Exemplo:

```
<input type="text" id="nome">  
<p id="resposta"></p>  
<script>  
  const input = document.getElementById("nome");  
  input.addEventListener("input", () => {  
    document.getElementById("resposta").textContent = `Olá, ${input.value}`;  
  });  
</script>
```

Eventos comuns:

- click, dblclick
- mouseover, mouseout
- keydown, keyup, input

Criando e Removendo Elementos

Criação:

```
const novo = document.createElement("div");  
novo.textContent = "Elemento criado";  
document.body.appendChild(novo);
```

Remoção:

```
const el = document.getElementById("paraRemover");  
el.remove();
```

Substituição:

```
elementoPai.replaceChild(novoElemento, antigoElemento);
```

Boas Práticas

- Prefira `querySelector()` para seleção flexível.
- Use `addEventListener()` ao invés de eventos inline.
- Agrupe lógicas em funções reutilizáveis.
- Para SPAs: o DOM é manipulado com Virtual DOM em React ou Template Reativo em Vue.

Exemplos práticos

- Os próximos slides são exemplos de aplicações do conteúdo apresentado.

document.querySelector()

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <title>Interatividade com addEventListener</title>
  <style>
    #outerdiv {
      padding: 50px;
      width: 100px;
      height: 100px;
      border: 1px solid black;
    }

    #innerdiv {
      width: 100%;
      height: 100%;
      border: 1px solid black;
      background: silver;
      transition: background 0.5s ease;
    }
  </style>
</head>
...
```

```
...
<body>
  <div id="outerdiv">
    <div id="innerdiv"></div>
  </div>

  <script>
    const outerdiv = document.querySelector('#outerdiv');
    const innerdiv = document.querySelector('#innerdiv');

    // Evento: mouse entra
    outerdiv.addEventListener('mouseover', () => {
      innerdiv.style.background = 'yellow';
    });

    // Evento: mouse sai
    outerdiv.addEventListener('mouseout', () => {
      innerdiv.style.background = 'silver';
    });
  </script>
</body>
</html>
```

Selecionar elementos com querySelectorAll

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <title>Manipulação DOM - Aula</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      padding: 20px;
    }
    h2, div, span {
      transition: background-color 0.5s ease;
      padding: 5px;
      display: inline-block;
      margin: 5px;
    }
    .highlight-h2 {
      background-color: lightblue;
    }
    .highlight-div {
      background-color: lightgreen;
    }
    .highlight-span {
      background-color: lightcoral;
    }
    button {
      margin-top: 20px;
      margin-right: 10px;
      padding: 10px;
      cursor: pointer;
    }
  </style>
</head>
```

```
<body>
  <h1>Um elemento H1</h1>
  <h2>Um elemento H2</h2>
  <div>Uma DIV</div>
  <p>Um elemento p.</p>
  <p>Um elemento p com um elemento <span style="color: brown;">span</span> dentro.</p>

  <button onclick="destacar()">Destacar Elementos</button>
  <button onclick="resetar()">Resetar Estilos</button>

  <script>
    function destacar() {
      document.querySelectorAll('h2').forEach(el => {
        el.classList.add('highlight-h2');
      });

      document.querySelectorAll('div').forEach(el => {
        el.classList.add('highlight-div');
      });

      document.querySelectorAll('span').forEach(el => {
        el.classList.add('highlight-span');
      });
    }

    function resetar() {
      document.querySelectorAll('h2').forEach(el => {
        el.classList.remove('highlight-h2');
      });

      document.querySelectorAll('div').forEach(el => {
        el.classList.remove('highlight-div');
      });

      document.querySelectorAll('span').forEach(el => {
        el.classList.remove('highlight-span');
      });
    }
  </script>
</body>
</html>
```

Exemplo - Alterar classe

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <title>Alternância de Classe</title>
  <style>
    .mystyle {
      background-color: aqua;
      padding: 30px;
      color: black;
      font-size: 18px;
      border-radius: 10px;
      transition: all 0.5s ease;
      box-shadow: 2px 2px 5px gray;
    }
    .mystyle2 {
      background-color: coral;
      padding: 50px;
      color: white;
      font-size: 22px;
      border-radius: 20px;
      transition: all 0.5s ease;
      box-shadow: 4px 4px 10px darkred;
    }
    button {
      padding: 10px 20px;
      font-size: 16px;
      cursor: pointer;
      margin-bottom: 20px;
    }
    p {
      font-weight: bold;
    }
  </style>
</head>
```

```
<body>

  <p>Clique no botão para alternar o estilo da DIV:</p>

  <button onclick="myFunction()" id="myBtn">Aplicar Estilo 2</button>

  <div id="myDIV" class="mystyle">
    Eu sou um elemento DIV com estilo dinâmico!
  </div>

  <script>
    function myFunction() {
      const element = document.getElementById("myDIV");
      const button = document.getElementById("myBtn");

      if (element.className === "mystyle") {
        element.className = "mystyle2";
        button.innerText = "Voltar para Estilo 1";
      } else {
        element.className = "mystyle";
        button.innerText = "Aplicar Estilo 2";
      }
    }
  </script>

</body>
</html>
```

Exemplo scroll

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <title>Scroll Interativo com Voltar ao Topo</title>
  <style>
    body {
      height: 1500px;
      font-family: Arial, sans-serif;
      padding: 20px;
    }
    #myP {
      position: fixed;
      top: 20px;
      left: 20px;
      background-color: lightgray;
      padding: 10px 15px;
      border-radius: 5px;
      box-shadow: 2px 2px 5px rgba(0,0,0,0.2);
      transition: background-color 0.4s ease;
    }
    .test {
      background-color: yellow !important;
      font-weight: bold;
    }
    #topBtn {
      display: none; /* Oculto por padrão */
      position: fixed;
      bottom: 30px;
      right: 30px;
      z-index: 100;
      background-color: #007bff;
      color: white;
      padding: 12px 20px;
      border: none;
      border-radius: 8px;
      font-size: 14px;
      cursor: pointer;
      box-shadow: 2px 2px 10px rgba(0,0,0,0.3);
      transition: opacity 0.3s ease;
    }
    #topBtn:hover {
      background-color: #0056b3;
    }
    .info {
      margin-top: 100px;
      font-size: 18px;
    }
  </style>
</head>
```

```
<body>
  <p id="myP">
    Quando você rolar mais de 15 pixels, esse parágrafo mudará de estilo!
  </p>
  <div class="info">
    <p>Role a página para baixo e veja a mágica acontecer!</p>
    <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit....</p>
    <p>Continue rolando... </p>
  </div>
  <!-- Botão Voltar ao Topo -->
  <button id="topBtn" onclick="voltarAoTopo()">↑ Voltar ao Topo</button>
  <script>
    const p = document.getElementById("myP");
    const topBtn = document.getElementById("topBtn");
    window.addEventListener("scroll", () => {
      const scrollTop = document.documentElement.scrollTop;
      // Alterna a classe do parágrafo
      if (scrollTop > 15) {
        p.classList.add("test");
      } else {
        p.classList.remove("test");
      }
      // Exibe ou esconde o botão de voltar ao topo
      if (scrollTop > 100) {
        topBtn.style.display = "block";
      } else {
        topBtn.style.display = "none";
      }
    });
    function voltarAoTopo() {
      window.scrollTo({
        top: 0,
        behavior: "smooth"
      });
    }
  </script>
</body>
</html>
```

Exemplo - zebraar tabela

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <title>Tabela Zebra Interativa</title>
  <style>
    table {
      border-collapse: collapse;
      width: 60%;
      margin: 20px auto;
    }

    th, td {
      border: 1px solid #ccc;
      padding: 10px;
      text-align: left;
    }

    .zebraon {
      background-color: #f2f2f2;
      transition: background-color 0.3s ease;
    }

    tbody tr:hover {
      background-color: #d0ebff !important;
      cursor: pointer;
    }

    .buttons {
      text-align: center;
      margin-top: 20px;
    }

    button {
      padding: 10px 15px;
      margin: 5px;
      font-size: 14px;
      cursor: pointer;
    }
  </style>
</head>
```

```
<body>
  <table class="zebra" id="tabelaVendas">
    <thead>
      <tr><th>Vendedor</th><th>Total</th></tr>
    </thead>
    <tbody>
      <tr><td>Manoel</td><td>12.300,00</td></tr>
      <tr><td>Joaquim</td><td>21.300,00</td></tr>
      <tr><td>Maria</td><td>13.200,00</td></tr>
      <tr><td>Marta</td><td>32.100,00</td></tr>
      <tr><td>Antonio</td><td>23.100,00</td></tr>
      <tr><td>Pedro</td><td>31.200,00</td></tr>
    </tbody>
  </table>
  <div class="buttons">
    <button onclick="alternarZebra()">Alternar Zebra</button>
  </div>
  <script>
    let zebraAtiva = true;

    function aplicarZebra() {
      document.querySelectorAll('.zebra tbody tr')
        .forEach((tr, i) => {
          tr.classList.toggle('zebraon', zebraAtiva && i % 2 === 1);
        });
    }

    function alternarZebra() {
      zebraAtiva = !zebraAtiva;
      aplicarZebra();
    }

    window.onload = aplicarZebra;
  </script>
</body>
</html>
```

Exemplo - checkbox

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <title>Seleção de Hobbies</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      padding: 20px;
    }

    fieldset {
      border: 1px solid #ccc;
      padding: 15px;
      width: 300px;
    }

    legend {
      font-weight: bold;
    }

    .resultado {
      margin-top: 20px;
      padding: 10px;
      background-color: #f0f8ff;
      border: 1px solid #aaa;
      border-radius: 5px;
    }
  </style>
</head>
```

```
<body>
  <form id="myform">
    <fieldset>
      <legend>Selecione seus hobbies:</legend>
      <label><input name="hobbies" type="checkbox" value="🎬 Filmes" /> Filmes</label><br />
      <label><input name="hobbies" type="checkbox" value="⚽ Esportes" /> Esportes</label><br />
      <label><input name="hobbies" type="checkbox" value="📖 Leitura" /> Leitura</label><br />
      <label><input name="hobbies" type="checkbox" value="😴 Dormir" /> Dormir</label><br /><br />
      <input type="submit" value="Enviar" />
    </fieldset>
  </form>
  <div class="resultado" id="resposta"></div>
  <script>
    document.querySelector('#myform').onsubmit = function (e) {
      e.preventDefault(); // Impede o envio padrão do formulário
      const selecionados = this.querySelectorAll('input[name="hobbies"]:checked');
      const saida = document.getElementById('resposta');
      if (selecionados.length === 0) {
        saida.innerHTML = "⚠ Nenhum hobby selecionado.";
        return;
      }
      let hobbiesEscolhidos = "<strong>Seus hobbies selecionados:</strong><ul>";
      selecionados.forEach(item => {
        hobbiesEscolhidos += `<li>${item.value}</li>`;
      });
      hobbiesEscolhidos += "</ul>";
      saida.innerHTML = hobbiesEscolhidos;
    };
  </script>
</body>
</html>
```

Exemplo temporizador

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <title>Notifier</title>
  <style>
    body { font-family: sans-serif; padding: 20px; }
    #msg p { background: #eef; padding: 10px; border-left: 5px solid #88f; }
    #notify { padding: 10px; font-size: 14px; }
  </style>
</head>
```

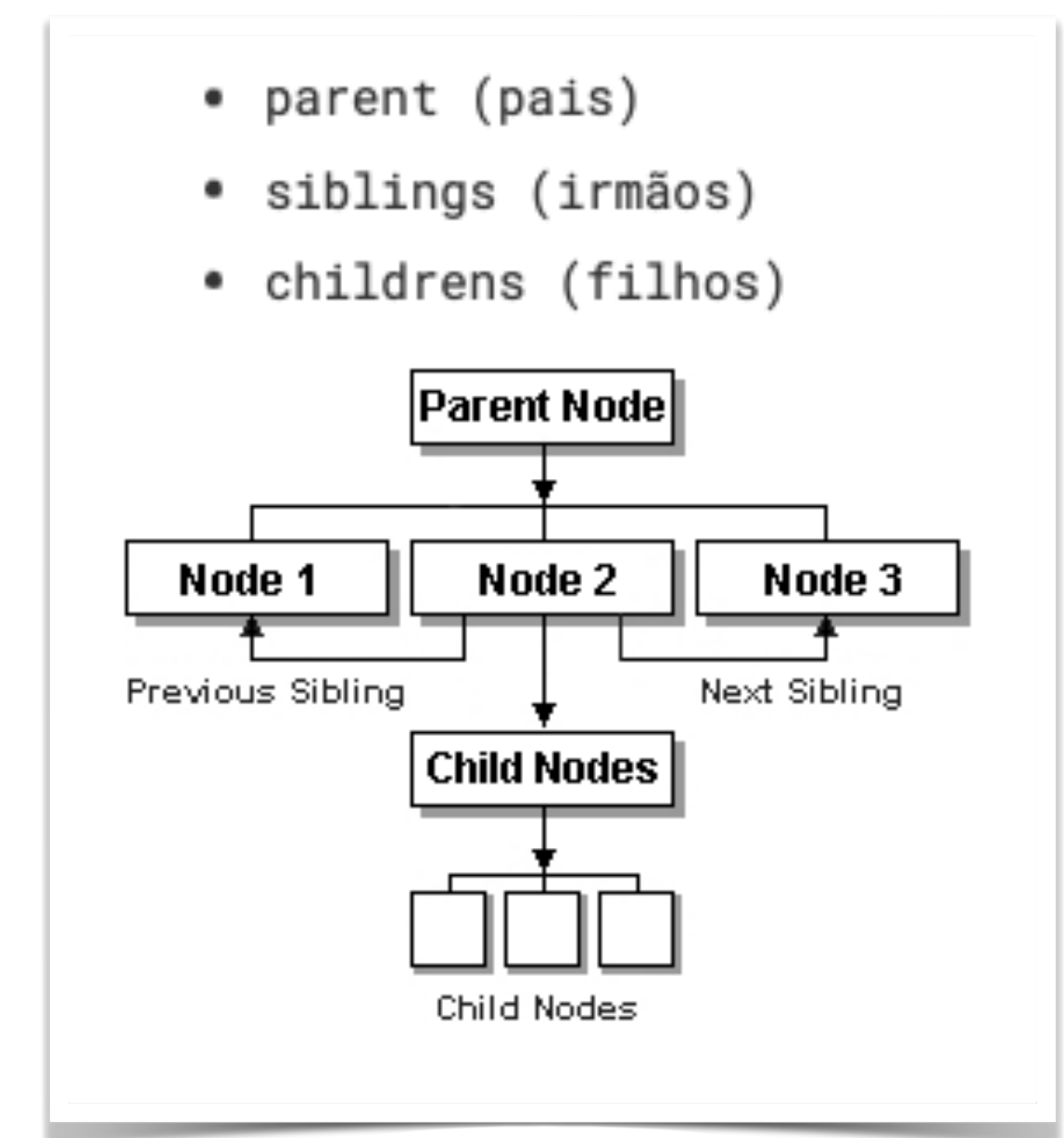
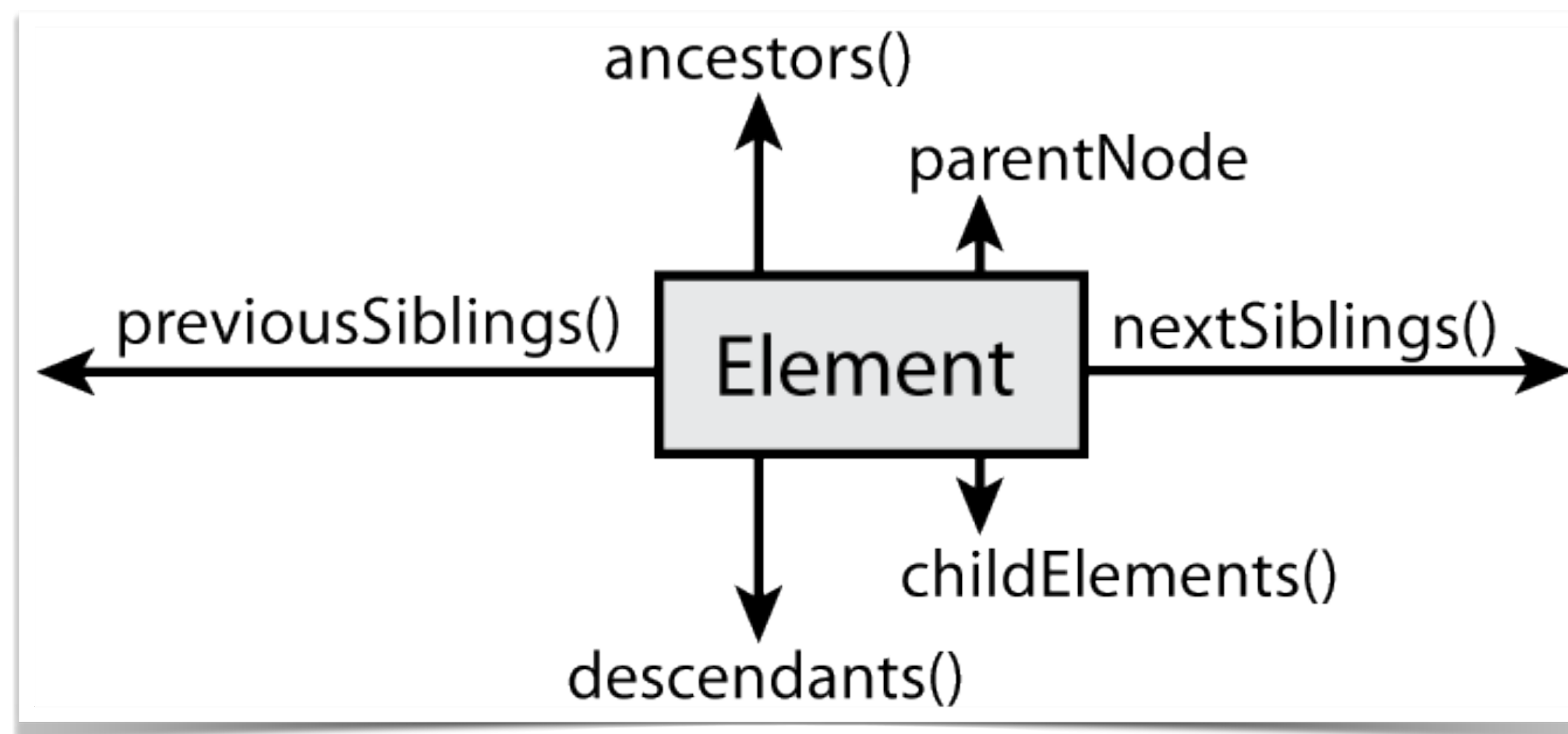
```
<body>
  <input type="button" id="notify" value="Notificação em 5 segundos" />
  <div id="msg"></div>
  <script>
    const btn = document.getElementById("notify");
    btn.onclick = () => {
      btn.disabled = true;
      notify("Você será notificado em 5 segundos...");
      setTimeout(() => {
        notify("✅ Ok! Notificação recebida.");
        btn.disabled = false;
      }, 5000);
    };
    function notify(text) {
      const p = document.createElement("p");
      p.textContent = text;
      document.getElementById("msg").appendChild(p);
      titleFlick();
    }
    function titleFlick() {
      if (document.hasFocus()) {
        document.title = "Notifier";
        return;
      }
      document.title = document.title === "Notifier" ? "*** Notifier" : "Notifier";
      setTimeout(titleFlick, 250); // Chama recursivamente enquanto fora de foco
    }
  </script>
</body>
</html>
```

Métodos usados para percorrer o documento

- hasChildNodes - retorna true se o elemento possui filhos;
- firstChild - retorna uma referência ao primeiro elemento;
- lastChild - retorna uma referência ao último elemento;
- **firstElementChild** - retorna uma referência ao primeiro elemento filho;
- **lastElementChild** - retorna uma referência ao último elemento filho;
- nextSibling - retorna qualquer elemento posterior;
- previousSibling - retorna qualquer elemento anterior;
- **nextElementSibling** - retorna uma referência ao irmão posterior ao elemento;
- **previousElementSibling** - retorna uma referência ao irmão anterior ao elemento;
- nodeName - retorna o nome da TAG do elemento (apenas para elementos nó);
- nodeValue - retorna o valor do elemento (apenas para elementos texto);
- nodeType - retorna o tipo do elemento;
- parentNode - retorna uma referência ao elemento pai.

Essas propriedades são a chave para ser capaz de manipular a árvore de documentos para acessar dados específicos do nó.

Navegando pela árvore DOM



Sugestão de vídeos

- Percorrendo a Árvore DOM - <https://www.youtube.com/watch?v=X3x0AZka5KY>
- Manipulação de DOM com JavaScript, do zero, na prática - <https://www.youtube.com/watch?v=UftSB4DaRU4>